

COL774 Assignment 1

Shrenik Sakala (2022MT11920)

September 5, 2025

Linear Regression

Batch Gradient Descent

In order to choose the optimal value for the learning rate, we first decide on a stopping criterion. It is a function of the change in the value of $J(\theta)$. We use a threshold of 10^{-12} on the square of the difference in loss functions for two consecutive values of parameters as our stopping criterion.

Convergence Criterion

$$(J(\theta^{(l+1)}) - J(\theta^{(l)}))^2 \leq \epsilon$$

Cost Function

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(y^{(i)} - \theta^T x^{(i)} \right)^2$$

To find the optimal learning rate, we tested a wide range of rates, from very small to large. We then plot the resulting final loss against each learning rate.

The best learning rate is found at the **"bottom" of the curve**, the point where the loss drops, just before it begins to increase due to divergence. This method quickly identifies the highest stable rate for the fastest training. The plot we get is as follows, and it is clear that optimal learning rate is 1:

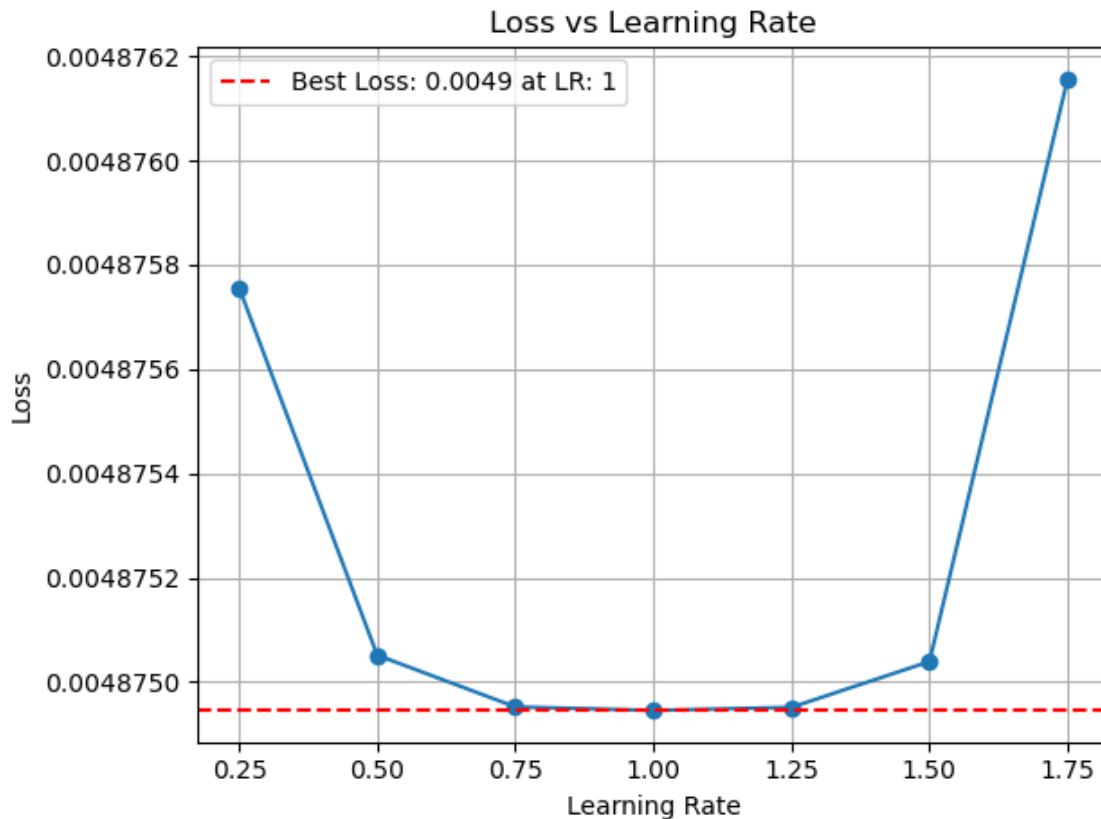


Figure 1: Choosing optimal learning rate

Final Results for learning rate 1:

- Tolerance: 1e-12
- Optimal theta: [6.21867801, 29.06475512]
- Final loss: 0.004874945275979409
- Theta history length: 4

Plots and Observations

2-D plot of Actual Data and Hypothesis Function

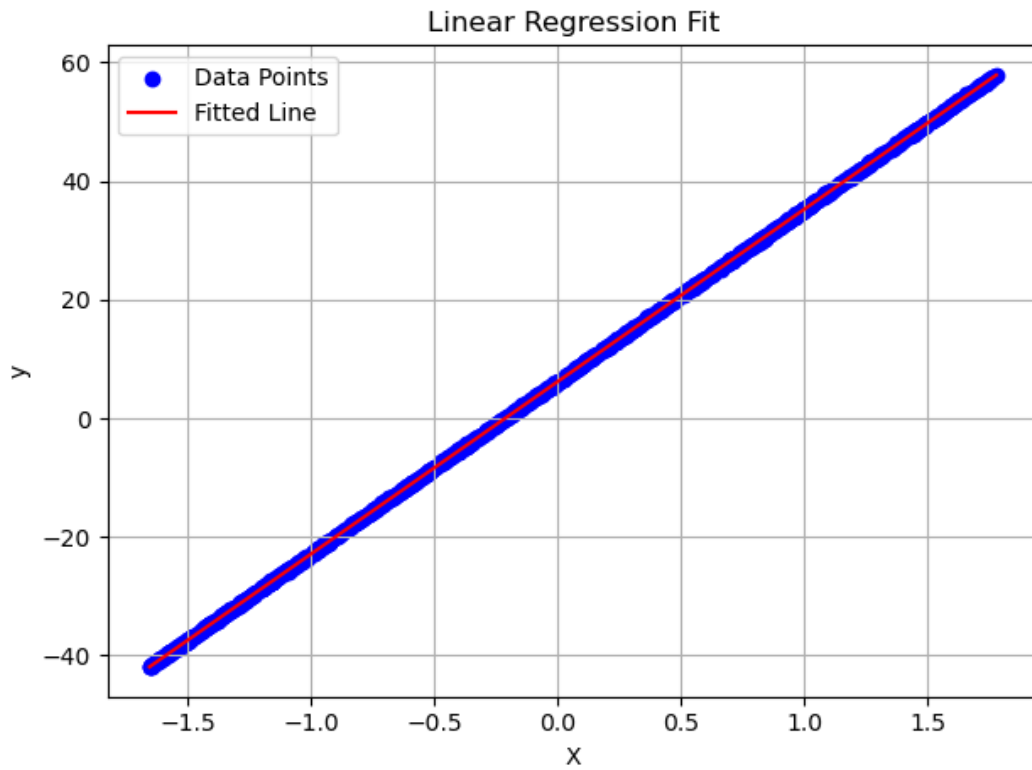


Figure 2: Linear Regression Line of Fit

3-D mesh for error value movement with learning rate 1

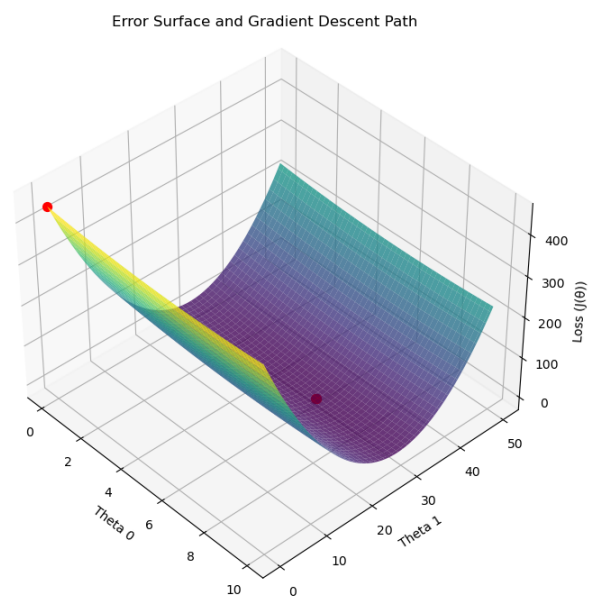


Figure 3: Error surface and Gradient Descent 3-D Mesh plot

Contour plots for different learning rates

Learning rate	Optimal theta0	Optimal theta1	Final Loss
1	6.218678	29.064755	0.004875
0.1	6.218093	29.061994	0.004879
0.025	6.217421	29.058831	0.004893
0.001	6.212106	29.033826	0.005374

Table 1: Learning rate vs Optimal Theta and Final Loss

Learning Rate = 1

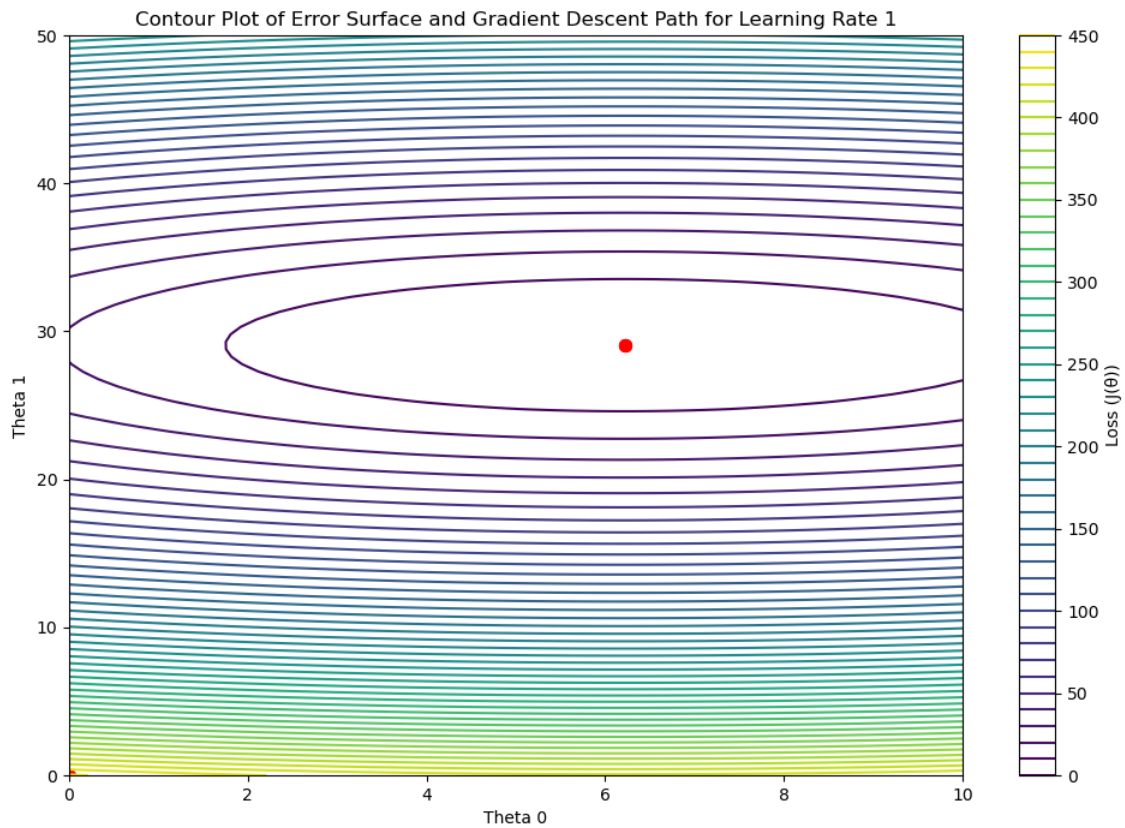


Figure 4: Contour plot for learning rate 1

Learning Rate = 0.1

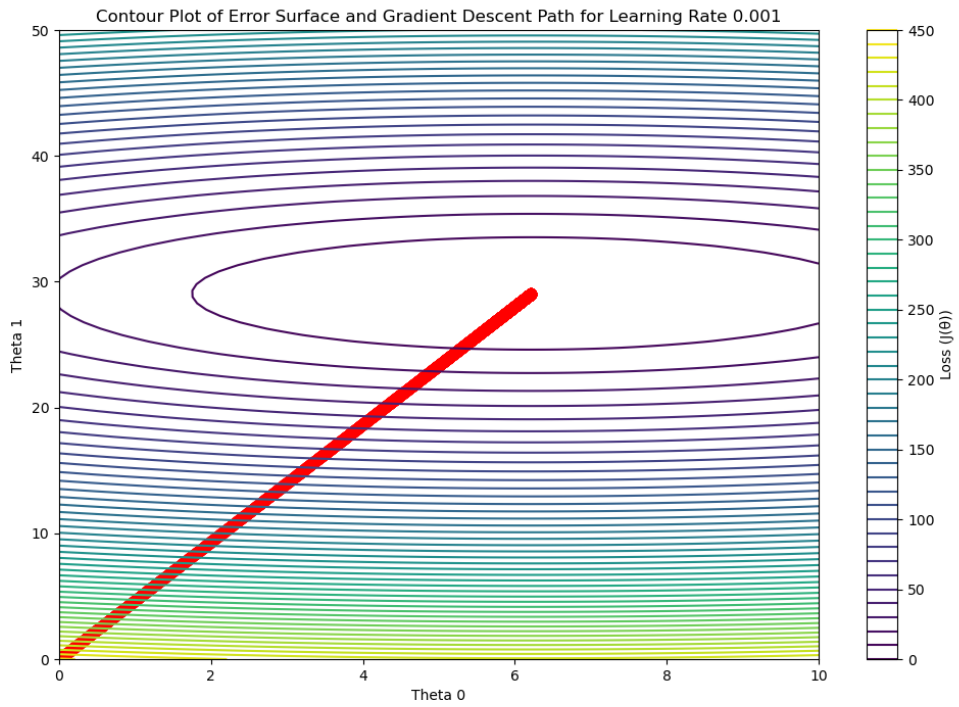


Figure 5: Contour plot for learning rate 0.1

Learning Rate = 0.025

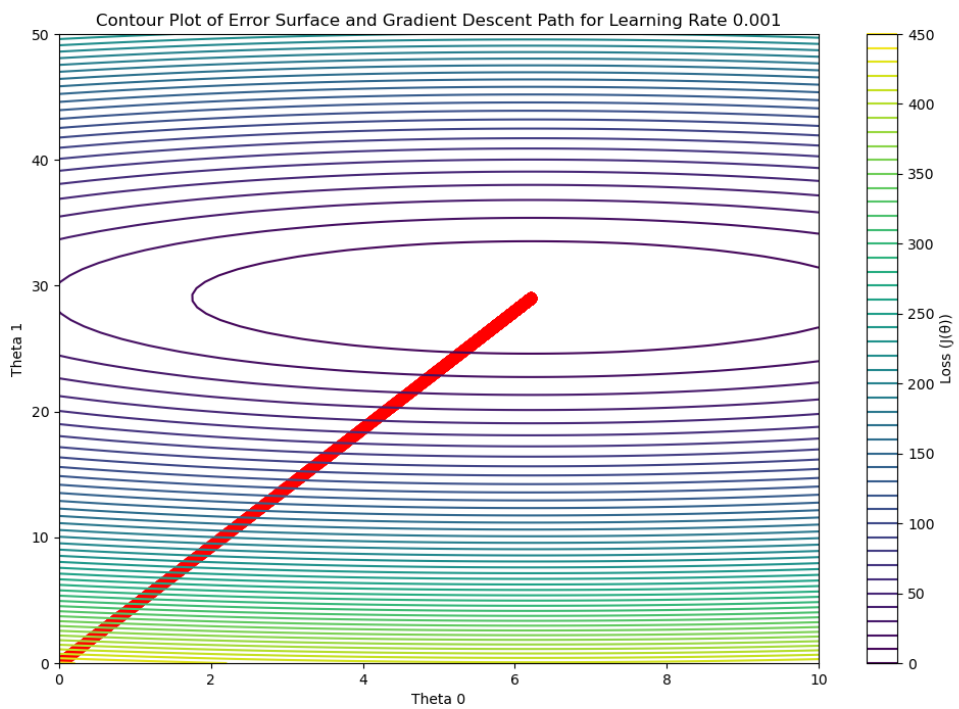


Figure 6: Contour plot for learning rate 0.025

Learning Rate = 0.001

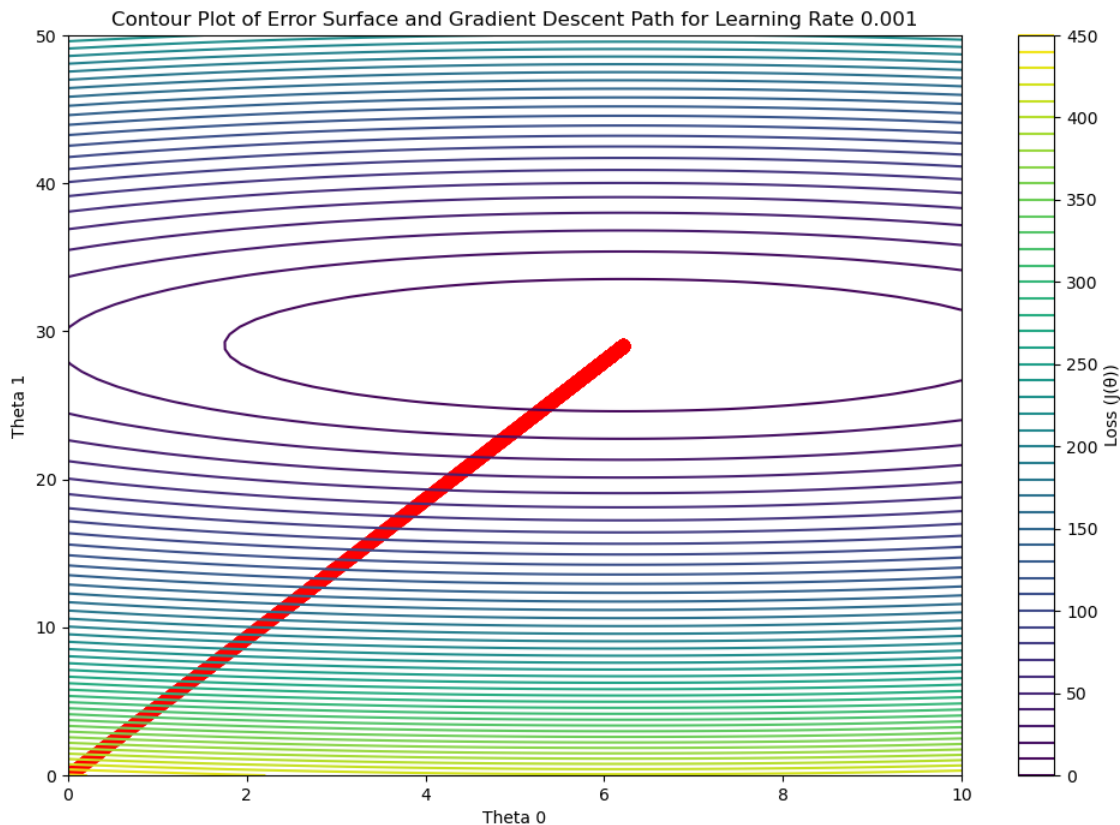


Figure 7: Contour plot for learning rate 0.001

Sampling, Closed Form and Stochastic Gradient Descent

Data generation and sampling

We generate synthetic training data. Consider a given hypothesis h_θ (i.e. known $\theta_0, \theta_1, \theta_2$) for a data point $x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$. Note that $x_0 = 1$ is the intercept term.

$$y = h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

Adding Gaussian noise, equation becomes

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \epsilon$$

where $\epsilon \sim \mathcal{N}(0, \sigma^2)$

Stochastic Gradient Descent

The training process is organized into epochs. At the beginning of each epoch, the entire dataset, which consists of m training examples, is randomly shuffled. This is an important step that helps prevent the model from getting stuck in local minima and improves its ability to generalize. The shuffled data is then processed in a round-robin fashion, with each pass over the data constituting a single epoch.

The data is divided into $\frac{m}{r}$ batches, with each batch containing r examples. For a given batch number b ($1 \leq b \leq \frac{m}{r}$), the set of examples is denoted as $\{x^{(i_1)}, x^{(i_2)}, \dots, x^{(i_r)}\}$, where the index $i_k = (b-1)r + k$. The loss function, computed as the mean squared error over these r examples, is given by:

$$J_b(\theta) = \frac{1}{2r} \sum_{k=1}^r (y^{(i_k)} - h_{\theta}(x^{(i_k)}))^2$$

For each batch, the gradient of the loss function is calculated and the model parameters θ are updated in the direction of the negative gradient, with a step size determined by the learning rate. This iterative process continues until the model's performance no longer improves, at which point the algorithm is terminated to prevent overfitting.

Termination Criterion for SGD

The loss function in Stochastic Gradient Descent (SGD) does not decrease monotonically but exhibits high-frequency noise due to the mini-batch sampling. A naive stopping criterion based on the change in raw loss between consecutive epochs can therefore lead to premature termination.

To create a more robust criterion, we monitor the convergence of a smoothed version of the loss function. We use the absolute difference between two consecutive simple moving averages (SMA) of the loss, each computed over a window of W epochs. The optimization process is terminated when this difference falls below a predefined tolerance, ϵ .

Let L_t be the loss at epoch t . The moving average at epoch T over a window of size W is:

$$\text{MA}_T = \frac{1}{W} \sum_{i=T-W+1}^T L_i$$

The algorithm terminates at epoch T if the following condition is met:

$$|\text{MA}_T - \text{MA}_{T-1}| < \epsilon$$

For our implementation, we used a window size $W = 10$ and a tolerance of $\epsilon = 10^{-4}$. This ensures that training halts only when the underlying trend of the loss function has flattened, indicating that the model has converged.

Varying Batch Size

Batch Size	Epochs	Theta	MSE in θ	Train MSE	Test MSE
800000	3311	$\begin{pmatrix} 1.87184976 \\ 1.2463759 \\ 1.91798528 \end{pmatrix}$	1.1576486754012059	2.36593992	2.36509957
8000	123	$\begin{pmatrix} 2.90564586 \\ 1.02060488 \\ 1.99351926 \end{pmatrix}$	0.09679496478943171	2.0019209	2.00138428
80	12	$\begin{pmatrix} 3.00487731 \\ 0.9963604 \\ 2.0041969 \end{pmatrix}$	0.007392482604732796	1.99933079	1.99895085
1	32	$\begin{pmatrix} 2.99647995 \\ 0.97131899 \\ 1.98680404 \end{pmatrix}$	0.0317667201043603	2.00882835	2.00917722

Table 2: Training Performance after varying batch sizes

Closed-form solution of linear regression

In linear regression, we seek to minimize the cost function $J(\theta)$, which represents the sum of squared errors in matrix form.

$$J(\theta) = \frac{1}{2} (X\theta - Y)^T (X\theta - Y)$$

To find the optimal parameters, we find where the gradient of the cost function is zero.

$$\nabla_{\theta} J(\theta) = 0$$

$$\frac{\partial}{\partial \theta} \left[\frac{1}{2} (\theta^T X^T X \theta - 2\theta^T X^T Y + Y^T Y) \right] = 0$$

Taking the derivative and setting to zero, we arrive at the normal equation.

$$X^T X \theta - X^T Y = 0$$

Solving for θ gives the closed-form solution.

$$\theta = (X^T X)^{-1} X^T Y$$

Optimal θ : $\begin{pmatrix} 3.00400433 \\ 0.99895449 \\ 2.00076897 \end{pmatrix}$ **MSE in θ :** 0.004209397024322285

Train MSE: 1.99915008, **Test MSE:** 1.99876829

Comparison of Performance

Impact of Batch Size on Model Accuracy

- **Very Large Batch Size (800000):** This approximates full-batch Gradient Descent. It performed poorly, yielding the highest error in both the parameter estimation (MSE in $\theta \approx 1.157$) and the final prediction error (Test MSE ≈ 2.365). This suggests it may have gotten stuck in a suboptimal local minimum or required a different learning rate.
- **Stochastic SGD (Batch Size = 1):** While much better than the large batch, this approach resulted in a higher parameter error (MSE in $\theta \approx 0.032$) compared to the mini-batch methods. The high variance from single-sample updates prevents the parameters from settling into the precise optimal values, leading to a slightly higher Test MSE of 2.009.
- **Mini-Batch (Batch Size = 80):** This batch size delivered the best performance. It achieved the lowest parameter error (MSE in $\theta \approx 0.0074$). This best parameter estimation led to the best predictive accuracy.

Comparison with the Closed-Form Solution

- The optimal Closed-Form solution has a Test MSE of 1.99876.
- The SGD model with a batch size of 80 achieved a nearly identical Test MSE of 1.99895.
- This observation demonstrates that a well-tuned mini-batch SGD can find a solution that is very close to the closed-form optimum.

Effect on Computational Efficiency and Convergence

The number of epochs required to reach the termination criteria highlights the trade-off between batch size and convergence speed.

- There is an inverse relationship between batch size and the number of epochs required. The larger the batch, the fewer updates are performed per epoch, thus requiring more epochs to converge.
- The batch size of 800000 was extremely inefficient, needing 3311 epochs.
- In contrast, the batch size of 80 was the most computationally efficient, converging in just 12 epochs.

Conclusion

The experimental results strongly indicate that mini-batch SGD (specifically with a size of 80 in this case) provides the optimal balance. It overcomes the noisy updates of pure SGD like batch size 1 and the slow, costly iterations of full-batch Gradient Descent like batch size 800000. It achieved the highest model accuracy, almost perfectly matching the closed-form solution, while also being the most computationally efficient in terms of epochs to convergence.

Plots of θ update paths for different batch sizes

Batch size = 800000

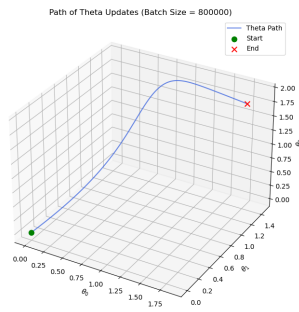


Figure 8: Path of θ updates for batch size 800000

Batch size = 8000

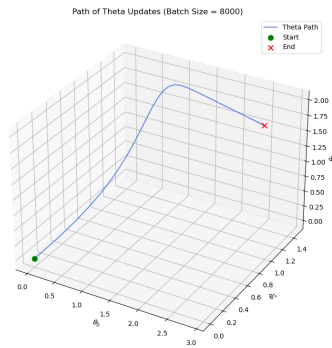


Figure 9: Path of θ updates for batch size 8000

Batch size = 80

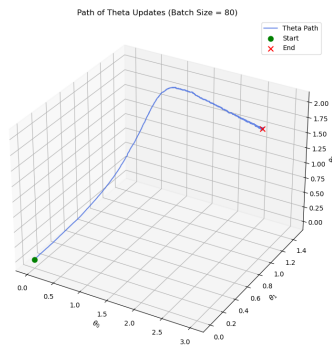


Figure 10: Path of θ updates for batch size 80

Batch size = 1

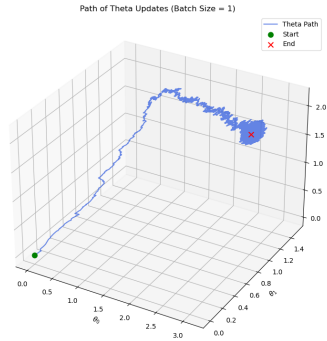


Figure 11: Path of θ updates for batch size 1

Conclusion

A small batch size (ex. 1) produces a noisy path. While exploring the parameter space, this high variance can lead to premature termination. Conversely, a large batch size (ex. 800000) results in a smooth convergence path. However, this stability comes at the cost of requiring a significantly higher number of epochs to reach the minimum.

Logistic Regression

The log-likelihood function for logistic regression is given by the equation:

$$LL(\theta) = \sum_{i=1}^m \left(y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right)$$

The gradient vector is computed as below:

$$\nabla_{\theta}(LL(\theta)) = X^T(h_{\theta}(X) - Y)$$

The Hessian matrix (the matrix of second partial derivatives) is given by:

$$H = \nabla_{\theta}^2 LL(\theta) = \sum_{i=1}^m h_{\theta}(x^{(i)})(1 - h_{\theta}(x^{(i)}))x^{(i)}(x^{(i)})^T$$

Newton's Method Update Rule

To maximize the log-likelihood, we need to find θ such that $\nabla_{\theta}LL(\theta) = 0$. Newton's method provides an iterative update rule to find this θ :

$$\theta^{(t+1)} := \theta^{(t)} - H^{-1}\nabla_{\theta}LL(\theta^{(t)})$$

where t is the iteration number.

Decision Boundary plot

Optimal theta after 10 iterations of Newton's Method: The calculated parameter vector, $\theta^{(10)}$, is approximately:

$$\theta^{(10)} \approx \begin{bmatrix} 0.4013 \\ 2.5885 \\ -2.7256 \end{bmatrix}$$

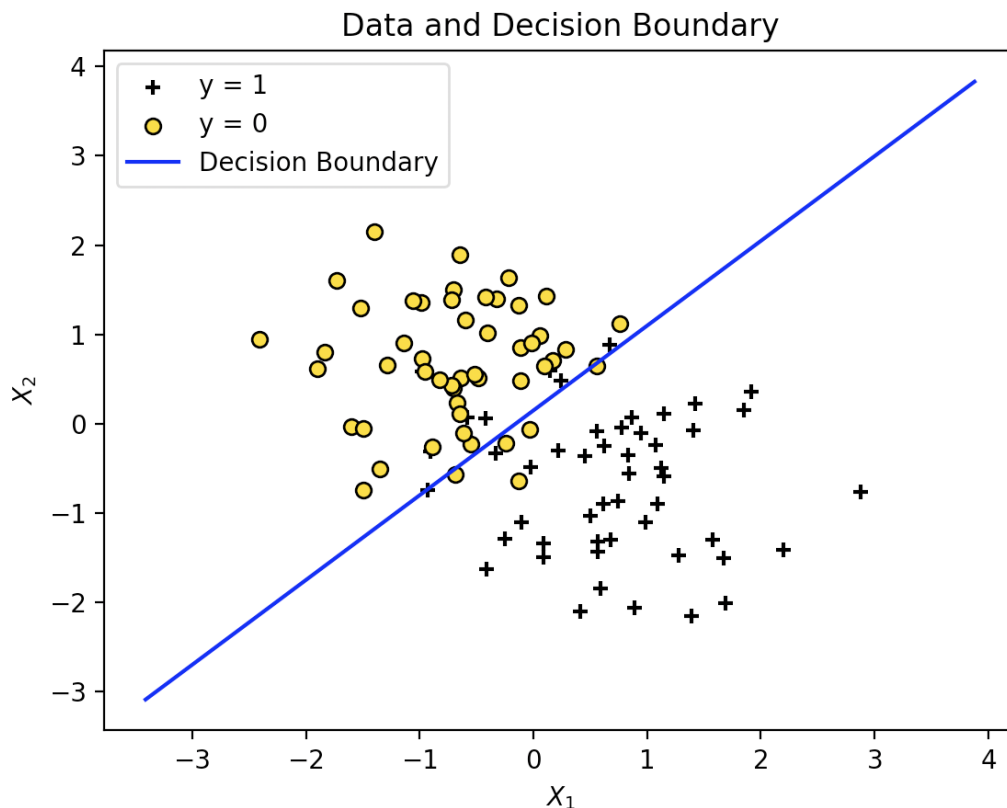


Figure 12: Decision Boundary based on our hypothesis

Gaussian Discriminant Analysis

GDA assuming same Σ

The decision boundary in Gaussian Discriminant Analysis (GDA) assuming same covariance matrix is the line where the posterior probabilities for each class are equal. Given the class means μ_0 and μ_1 , and a shared covariance matrix Σ , the equation for this boundary is:

$$(\mu_1 - \mu_0)^T \Sigma^{-1} \mathbf{x} - \frac{1}{2}(\mu_1^T \Sigma^{-1} \mu_1 - \mu_0^T \Sigma^{-1} \mu_0) + \log \left(\frac{P(y=1)}{P(y=0)} \right) = 0$$

where $P(y=k)$ is the prior probability of class k .

Results:

Mean μ_0 (Canada): $\begin{pmatrix} 0.75529433 \\ -0.68509431 \end{pmatrix}$

Mean μ_1 (Alaska): $\begin{pmatrix} -0.75529433 \\ 0.68509431 \end{pmatrix}$

Covariance matrix Σ : $\begin{bmatrix} 0.42953048 & -0.02247228 \\ -0.02247228 & 0.53064579 \end{bmatrix}$

Plot:

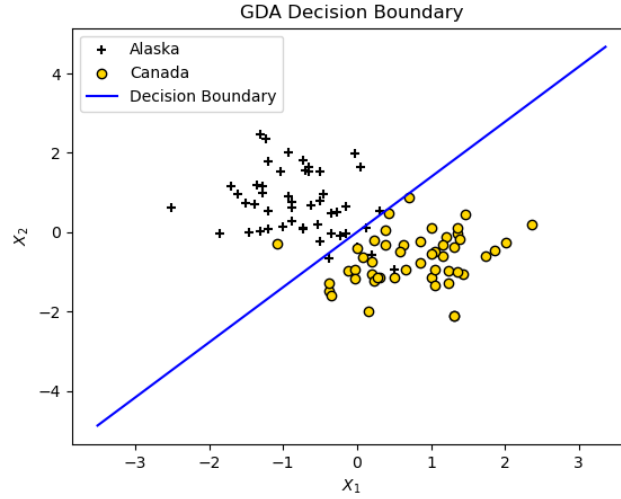


Figure 13: Decision Boundary for GDA assuming same covariance matrices

General GDA

The decision boundary in General Gaussian Discriminant Analysis (GDA) arises when each class k is modeled with its own mean μ_k and covariance matrix Σ_k . The boundary is the set of points $\mathbf{x}$ where the posterior probabilities are equal. Because the covariance matrices are not the same, the quadratic terms do not cancel, yielding a curved, quadratic surface. The equation for this boundary is:

$$(\mathbf{x} - \mu_1)^T \Sigma_1^{-1} (\mathbf{x} - \mu_1) - (\mathbf{x} - \mu_0)^T \Sigma_0^{-1} (\mathbf{x} - \mu_0) + \log \left(\frac{|\Sigma_1|}{|\Sigma_0|} \right) - 2 \log \left(\frac{P(y=1)}{P(y=0)} \right) = 0$$

Results:

Mean μ_0 (Canada): $\begin{pmatrix} 0.75529433 \\ -0.68509431 \end{pmatrix}$

Mean μ_1 (Alaska): $\begin{pmatrix} -0.75529433 \\ 0.68509431 \end{pmatrix}$

Covariance matrix Σ_0 (Canada): $\begin{bmatrix} 0.47747117 & 0.1099206 \\ 0.1099206 & 0.41355441 \end{bmatrix}$

Covariance matrix Σ_1 (Alaska): $\begin{bmatrix} 0.38158978 & -0.15486516 \\ -0.15486516 & 0.64773717 \end{bmatrix}$

Plot:

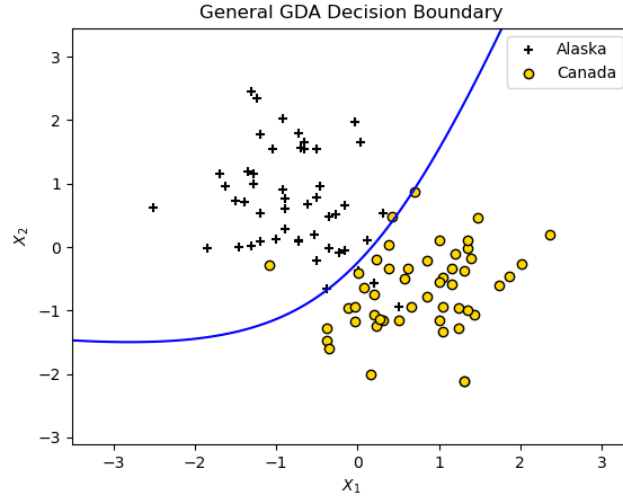


Figure 14: Decision Boundary for General GDA

Comparison plot

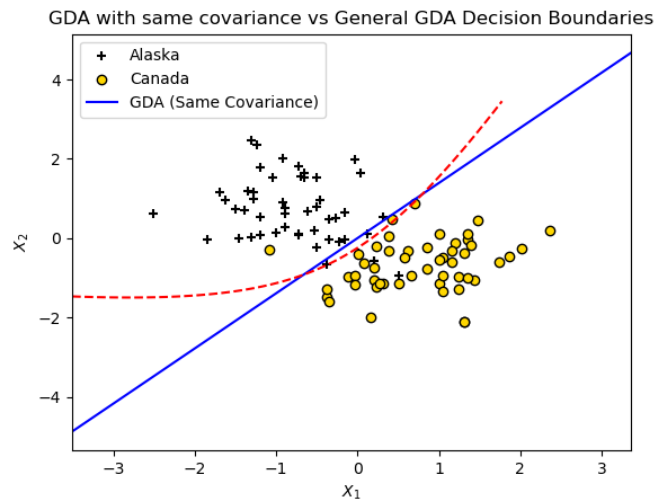


Figure 15: Comparing the decision boundaries of the two GDA approaches

Observations

- **Linear Boundary (GDA assuming same Σ):** This boundary is a rigid line, resulting from the assumption of a shared covariance matrix (Σ). Its simplicity gives it **high bias** but **low variance**, making it robust against overfitting and a good choice for smaller datasets.
- **Quadratic Boundary (General GDA):** This is a flexible, curved boundary that results from each class having its own covariance matrix (Σ_k). Its flexibility gives it **low bias**, allowing it to capture more complex patterns. However, this comes at the cost of **high variance**, making it susceptible to overfitting.
- **Conclusion:** The choice between them is a classic example of the **bias-variance trade-off**. General GDA is more complex but requires more data to be effective, whereas GDA assuming same Σ is a simpler, more constrained model.